

VERSION 2.4.2 · MIT · 100% LOCAL · ZERO TELEMETRY

Text Watermark Studio

The local forensics lab for AI text and file watermarks

Detect AI watermarks. Set your own. Deliver proof you can defend — instead of "looks like AI" feelings.

1,113
TESTS

15+
CLI COMMANDS

14×
SPEEDUP

8
PLUGINS

3
SIGNATURES

github.com/mrmixx-max/text-watermark-studio · MIT License

`pip install text-watermark-studio`

Everything you need for AI watermark forensics

Text Watermark Studio covers the entire workflow — from detection through proof generation to active marking. No cloud. No telemetry. No dependencies.

**KGW Detection with Z-Score**

Statistical detection of greenlist watermarks. $Z \geq 4.0$ = watermark_detected. Multi-key with Bonferroni correction.

**Multi-Bit Payload Watermarking**

Invariant-feature codebook (Yoo et al., ACL 2023). Embed/extract user IDs, timestamps, run IDs.

**HMAC + ML-DSA Signatures**

Sign forensic reports (HMAC-SHA256) or post-quantum (ML-DSA FIPS 204). Verifiable, auditable.

 **ΔZ Measurement & Proof**

Measure watermark strength before and after an attack. With receipt — not just claim.

**Remove & Rewrite Engine**

Best-effort watermark removal: clean → dilute → structural rewrite. Honest about what changes.

**Findings Report (C5)**

AI explanation report with evidence classes A-D, priority 0-5, signed. For courts and expert opinions.

**Web Dashboard (HTMX)**

5 pages: Overview, Watermark, Files, Reports, Settings. SSE real-time stats. No build step.

**Multi-Model LLM Router**

Fallback, round-robin load balancing, health checks. Automatic on Ollama model failure.

**Prometheus + Grafana**

8 metrics, alerting rules, Docker Compose stack. Production monitoring out of the box.

**Timing-Safe Security**

hmac.compare_digest, JWT auth, rate limiting (100/min), API key rotation (5 keys), SSRF prevention.

**Batch & Watch Mode**

Batch embed with --verify ($Z > 4$ confirmation). Directory watching with KGW text detection.

**--verify Flag**

File-clean inspects the cleaned bytes again. Proves the strip worked.

**--quiet Flag**

Machine-readable output for scripting/pipelines. Status messages only on stderr.

**Code/Audio/Video Plugins**

Detect AI-generated code, SynthID-style audio metadata, C2PA/XMP in video containers.

Three layers. Three signatures. One answer.

Detection Layers

Layer 1 — Invisible Characters

- ✓ Bidirectional controls (LRE, RLE, LRO, RLO)
- ✓ Zero-width spaces, joiners, tag blocks
- ✓ Format characters, Unicode steganography

Layer 2 — Style Markers

- ✓ 18 AI phrasing patterns + inflected forms
- ✓ Statistical fingerprints in word choice

Layer 3 — Statistical Watermarks

- ✓ KGW Greenlist (Kirchenbauer et al.)
- ✓ E-Process (anytime-valid, early stopping)
- ✓ Signature filter (FPR control, arXiv 2606.18430)

KGW Detection

```
ai-wm detect article.txt --key demo-kgw-1
# Z = 15.9 → watermark_detected

ai-wm detect article.txt --key wrong-key
# Z = -0.2 → no_signal

ai-wm delta-z ORIGINAL.txt --transform clean --key K
# ΔZ: 15.9 → 0.4 → watermark degraded
```

Signature Algorithms

HMAC-SHA256 (Default)

Zero deps. stdlib-only. For internal auditing.

ML-DSA-44 / 65 / 87 (FIPS 204)

Post-quantum. For reports that need to remain valid in 2030+. Requires cryptography≥50.

Security Hardening

- ✓ Timing-safe API key comparison (hmac.compare_digest)
- ✓ SSRF prevention via URL scheme validation
- ✓ Rate limiting (100 req/min per API key)
- ✓ JWT token auth with scopes
- ✓ API key rotation (5 active keys, expiry)
- ✓ Request logging middleware

File Formats

```
ai-wm file-inspect report.pdf
# {found, valid, key_id, algorithm, provenance}

ai-wm file-clean photo.jpg -o clean.jpg --verify
# C2PA, EXIF, XMP — verified_clear | residual_hard_bound

ai-wm file-embed doc.pdf --key K -o signed.pdf
# HMAC provenance mark in 8 formats
```

Every command now prints its output

Previously, many handlers ended with `else: pass`. Now every command prints JSON or `--pretty` output. Exit codes: 0 clean, 1 findings, 2 error.

Core Commands

```
ai-wm detect [file] --key K --level word|bpe # KGW + Unicode
+ Phrasing ai-wm clean [file] --nfkc --fold-confusables #
Remove Unicode Layer ai-wm dilute [file] --intensity
light|standard|aggressive # Rewrite phrases (33 rules) ai-wm
embed [file] --key K --gamma 0.25 # Set greenlist mark ai-wm
pipeline [file] --rewrite-mode #
detect→clean→dilute→rewrite→detect ai-wm remove [file] --use-
llm --aggressive # Best-effort removal
```

New Commands (v2.4.2)

```
ai-wm delta-z FILE --transform clean --key K # ΔZ measurement
with receipt ai-wm finding FILE --frs --sign SECRET # AI
report (C5), signed ai-wm trace FILE --window 500 --step 250
# Z-score trajectory ai-wm payload embed FILE --payload
"user-42" # Multi-bit watermarking ai-wm evade FILE --target-
z 3.9 # Adversarial evaluation
```

File Commands

```
ai-wm file-inspect FILE # Detect C2PA/EXIF/XMP ai-wm file-
clean FILE --verify # Clean + re-inspect ai-wm file-embed
FILE --key K # Set HMAC provenance ai-wm file-detect FILE --
key K # Verify provenance ai-wm report FILE --key K --pdf #
HTML/PDF forensic report
```

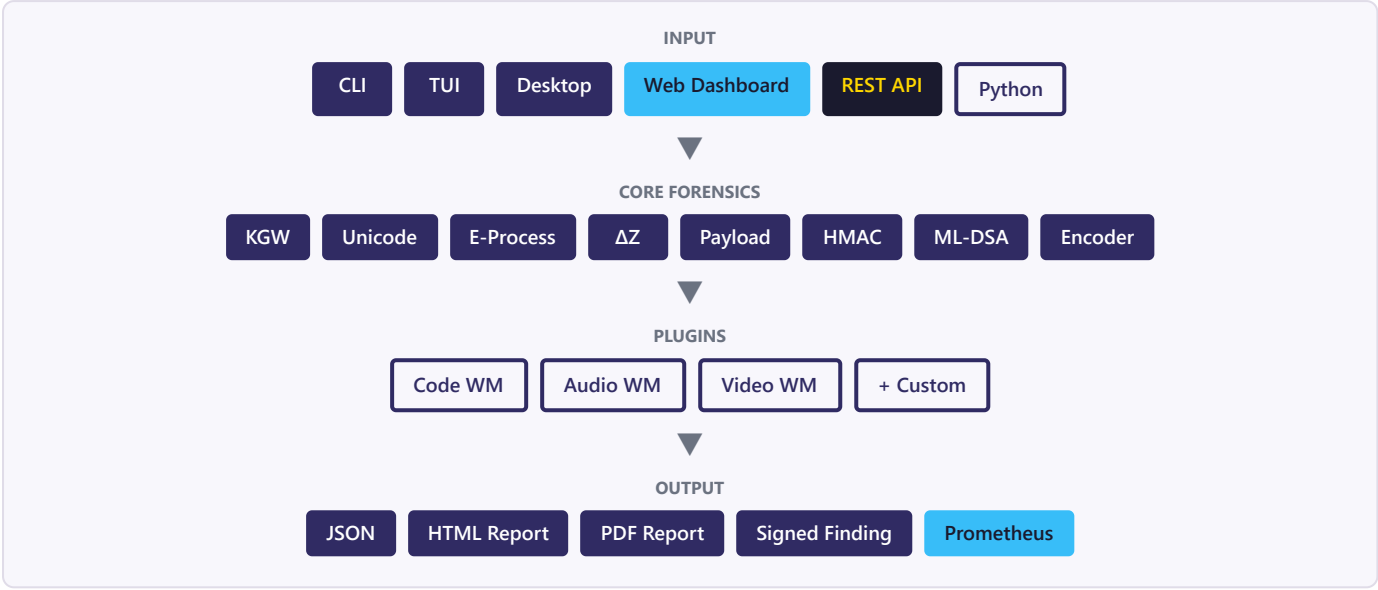
Report Signing

```
ai-wm report-keygen --algorithm mldsa-44 # Generate ML-DSA
key pair ai-wm report-sign FINDING.json --secret S # HMAC-
SHA256 or ML-DSA ai-wm report-verify SIGNED.json # Verify
signature (0/1/2)
```

Service Commands

```
ai-wm serve --port 8080 # FastAPI server (22 modules) ai-wm
dashboard --port 8080 # Web dashboard (HTMX) ai-wm tui #
Terminal UI (25 actions) ai-wm watch DIR --kgw # Watch
directory ai-wm batch IN OUT --verify # Batch with Z>4
confirmation
```

Plugins, Monitoring, Interfaces — One Codebase



Monitoring Stack

```
docker compose -f docker-compose.monitoring.yml up -d
# Prometheus :9090 · Grafana :3000 · Alerting

Metrics: http_requests_total · watermark_operations_total · detection_score_histogram · verdict_total · stream_pending · dlq_depth
· dlq_replays_total
```

API Modules (22 Routes)

Forensics · Metadata · Documents · PDF · RAG · LLM · Routing · Prompts · Optimization · Multi-Agent · Graph · Community · Rewrite · Export · Cloud-Upload · Queue/Streams · Auth · Health · Metrics

We tell you what we can't do.

Every technology has limits. We document them honestly instead of hiding them. That's the difference between a forensics lab and an "AI detector" that just does style matching.

What we can do

- ✓ Measure statistical watermarks with Z-score (reproducible, auditable)
- ✓ Set and verify our own watermarks
- ✓ Clean file metadata and prove it worked
- ✓ Sign reports (HMAC or ML-DSA) and verify them
- ✓ Detect encoding attacks (overlong UTF-8, homoglyphs, hidden BOMs)

What we can't do (and say honestly)

- ✓ Detect every AI text — only with the right method and the right key
- ✓ Remove statistical watermarks — only rewrite (degrade signal)
- ✓ Detect SynthID pixels without the 220MB research checkpoint
- ✓ Prove files without metadata context — only with your own registry

Comparison: Us vs. "AI Detectors"

Feature	TWS	Typical "AI Detector"
Local execution	✓ Yes	✗ Cloud
Reproducible score	✓ Z-Score	✗ Blackbox
Own watermarks	✓ KGW	✗ No
Signed proofs	✓ ML-DSA	✗ No
Honest limits	✓ Yes	✗ Promises
Open source (MIT)	✓ Yes	✗ Proprietary

Installation in 30 seconds

Python 3.10+. No cloud. No API key. No telemetry.

```
pip install text-watermark-studio
```

Source: github.com/mrmixx-max/text-watermark-studio
MIT License · Docs in docs/ · Issues welcome